



Art Institute Explorer - v2.0

Un buscador inteligente de obras de arte que permite a los usuarios explorar contenido del Art Institute of Chicago con autenticación, obra de arte sorpresa diaria y gestión de estado global avanzada.



Estado Actual - v2.0

Rama: v2_ArtApp

Fase Completada: Fase 1 - Redux Setup & Store

Fase Actual: Fase 2 - Error Boundaries & Componentes Auth



Objetivos v2.0

La versión 2.0 introduce mejoras arquitectónicas y nuevas funcionalidades diseñadas para un proyecto de nivel **Junior/Aprendiz**:

Mejoras Técnicas

- **Estado Global con Redux** - Centralizar estado con Redux Toolkit
- **Redux DevTools** - Debugging avanzado del estado
- **Error Boundaries** - Captura y manejo de errores en componentes
- **React Hooks** - Uso correcto de useState, useEffect, useCallback, useRef
- **React Hook Form** - Validación de formularios simplificada

Nuevas Funcionalidades

- **Sistema de Autenticación** - Registro e inicio de sesión
- **Obra de Arte Diaria** - Cada usuario logueado recibe una obra sorpresa al día
- **Optimización de Imágenes** - Lazy loading y múltiples resoluciones
- **Rutas Protegidas** - Acceso limitado a usuarios autenticados

PROF



Características v1.0 (Mantienen)

- **Búsqueda Semántica:** Busca obras de arte por términos, artistas, técnicas y períodos
- **Filtros Dinámicos Avanzados:**
 - Búsqueda por palabras clave
 - Filtrado por artista
 - Filtrado por material/técnica
 - Rango de fechas (año de creación)
 - Filtrado por cultura
- **Galería Visual Fluida:** Interfaz responsive con animaciones suaves
- **Vista Detallada:** Modal con información completa de cada obra
- **Paginación Inteligente:** Navegación fácil entre páginas de resultados

- **Diseño Responsive:** Funciona perfectamente en desktop, tablet y móvil

Requisitos Previos

- Node.js 16+
- npm o yarn
- Conexión a Internet (para consumir la API del Art Institute)

Instalación

1. Clonar el repositorio

```
git clone https://github.com/AnaLauDB/Art-API.git
cd Art-API
```

2. Instalar dependencias

```
npm install
```

3. Iniciar el servidor de desarrollo

```
npm run dev
```

La aplicación estará disponible en <http://localhost:5173>

Estructura del Proyecto v2.0

```
src/
├── components/
│   ├── ErrorBoundary.jsx           # [FASE 2] Captura global de
errores
│   ├── Auth/                       # [FASE 3] Componentes de
autenticación
│   │   ├── LoginForm.jsx
│   │   ├── RegisterForm.jsx
│   │   ├── ProtectedRoute.jsx
│   │   └── AuthModal.jsx
│   ├── Gallery/                   # Componentes de galería
(Mejorados v2.0)
│   │   ├── ArtworkCard.jsx        # [FASE 4] Con lazy loading de
imágenes
│   │   ├── ArtworkDetail.jsx
│   │   └── ArtworkGrid.jsx
│   └── DailyPick/                 # [FASE 5] Obra sorpresa diaria
```

```

|   |   |   | DailyArtwork.jsx
|   |   |   | DailyArtwork.css
|   |   |   | Search/
|   |   |   | |   | SearchBar.jsx          # Con useRef para input
|   |   |   | |   | FilterPanel.jsx
|   |   |   | Shared/
|   |   |   | |   | Header.jsx            # [NUEVO] Componentes compartidos
|   |   |   | |   | Navigation.jsx        # Con info de usuario logueado
|   |   |   | SearchBar.jsx              # v1.0 (se migrará)
|   |   |   | FilterPanel.jsx            # v1.0 (se migrará)
|   |   |   | ArtworkCard.jsx            # v1.0 (se migrará)
|   |   |   | ArtworkGrid.jsx            # v1.0 (se migrará)
|   |   |   | ArtworkDetail.jsx          # v1.0 (se migrará)
|   |   |   |
|   |   |   | services/
|   |   |   | |   | arteServices.js        # Llamadas API (Mantiene)
|   |   |   | |   | authServices.js       # [FASE 3] Lógica de
autenticación
|   |   |   | |   | imageServices.js      # [FASE 4] Optimización de
imágenes
|   |   |   |
|   |   |   | redux/
|   |   |   | |   | store.js
|   |   |   | |   | slices/
|   |   |   | |   | |   | authSlice.js    # Estado de autenticación
|   |   |   | |   | |   | artworksSlice.js # Estado de obras de arte
|   |   |   | |   | |   | dailyPickSlice.js # Estado de obra diaria
|   |   |   | |   | |   | filterSlice.js    # Estado de filtros
|   |   |   | |   | |   | uiSlice.js        # Estado de UI (modales,
notificaciones)
|   |   |   |
|   |   |   | utils/
|   |   |   | |   | constants.js           # URLs, configuraciones
|   |   |   | |   | dateUtils.js          # Funciones de fecha para obra
diaria
|   |   |   |
|   |   |   | styles/
|   |   |   | |   | SearchBar.css
|   |   |   | |   | FilterPanel.css
|   |   |   | |   | ArtworkGrid.css
|   |   |   | |   | ArtworkDetail.css
|   |   |   | |   | variables.css          # [NUEVO] CSS variables globales
|   |   |   |
|   |   |   | App.jsx                      # [FASE 6] Se migrará a Redux
|   |   |   | App.css
|   |   |   | index.css
|   |   |   | main.jsx                    # ✓ Con Redux Provider
|   |   |   | assets/

```

PROF

Fases de Implementación

Fase	Estado	Contenido
Fase 1	✅ Completa	Redux Setup, Store, Slices, DevTools
Fase 2	🔄 En Progreso	Error Boundaries (Global, Regional, Local)
Fase 3	⌚ Pendiente	Auth Services, LoginForm, RegisterForm, ProtectedRoute
Fase 4	⌚ Pendiente	Image Services, Lazy Loading, Optimización
Fase 5	⌚ Pendiente	DailyArtwork, Hook para obra diaria
Fase 6	⌚ Pendiente	Migrar App.jsx a Redux, Testing

🎓 Conceptos y Tecnologías por Fase

Fase 1 ✅ - Redux Setup

- **Redux Toolkit** - Simplifica configuración de Redux
- **Redux DevTools** - Debugging del estado global
- **Slices** - Estructura modular del estado
- **Reducers y Actions** - Cómo modificar el estado

Aprendizajes:

- Centralizar estado global vs estado local
- Separar lógica en slices según dominio
- Usar DevTools para inspeccionar cambios de estado

Fase 2 🔄 - Error Boundaries

- **Class Components** - Error Boundaries (requieren clase)
- **getDerivedStateFromError()** - Captura de errores
- **componentDidCatch()** - Logging de errores
- **Try/Catch** - Para errores en funciones async

Aprendizajes:

- Diferencia entre Error Boundaries y Try/Catch
- Niveles de Error Boundaries (Global, Regional, Local)
- Fallback UI para mejorar UX

Fase 3 ⌚ - Autenticación

- **React Hook Form** - Formularios validados
- **localStorage** - Persistencia de sesión
- **useSelector/useDispatch** - Interacción con Redux
- **ProtectedRoute** - Componentes condicionales

Aprendizajes:

- Validación de formularios sin librerías pesadas

- Estructura de componentes protegidos
- Gestión de autenticación con localStorage

Fase 4 ⌚ - Optimización de Imágenes

- **IIIF Protocol** - URLs dinámicas de imágenes
- **Lazy Loading** - Cargar imágenes cuando se necesitan
- **Responsive Images** - srcSet para múltiples resoluciones
- **imageServices.js** - Centralizar lógica de imágenes

Aprendizajes:

- Optimizar performance de galería
- Trabajar con APIs de imágenes externas
- Progressive image loading

Fase 5 ⌚ - Obra Diaria

- **useState + useEffect** - Lógica compleja en componentes
- **dateUtils.js** - Funciones de fecha
- **localStorage + Redux** - Cachear datos diarios
- **Aleatoriedad** - Seleccionar obra random cada día

Aprendizajes:

- Cómo cachear datos por fecha
- Sincronizar localStorage con Redux
- Lógica de "obra del día" sin backend

Fase 6 ⌚ - Migración Final

- **useCallback** - Optimizar re-renders
- **useRef** - Acceso directo al DOM
- **Thunks** - Acciones asincrónicas en Redux
- **Testing** - Validar componentes

Aprendizajes:

- Cuándo usar cada hook
- Performance optimization
- Testing en React

Art Institute of Chicago API v1

- Documentación: <https://api.artic.edu/docs/>
- Base URL: <https://api.artic.edu/api/v1>

Endpoints Principales

- [/artworks/search](#) - Búsqueda de obras con filtros

- `/artworks/{id}` - Detalles de una obra específica
- `/artists` - Lista de artistas
- `/artworks/search?facets=medium_display` - Agregaciones de medios

Uso

Búsqueda Simple

1. Ingresa un término en la barra de búsqueda (ej: "paisaje", "Van Gogh", "óleo")
2. Haz clic en "Buscar" o presiona Enter
3. Los resultados se mostrarán en la galería

Filtros Avanzados

1. Usa el panel de filtros en la izquierda para aplicar criterios específicos
2. Combina múltiples filtros para refinar resultados
3. Haz clic en "Aplicar filtros" para actualizar la búsqueda
4. Usa "Limpiar filtros" para restablecer

Ver Detalles

1. Haz clic en cualquier obra de arte en la galería
2. Se abrirá un modal con información completa
3. Haz clic en "Ver en el Art Institute" para más información
4. Cierra el modal con el botón X

Tecnologías Utilizadas v2.0

Core

- **React 19.2.5** - Librería de UI
- **Vite 8.0.10** - Build tool y dev server
- **JavaScript ES6+** - Lenguaje de programación

PROF

Estado Global

- **Redux Toolkit**  - Gestión de estado simplificada
- **React-Redux**  - Integración React + Redux
- **@redux-devtools/extension**  - DevTools para debugging

Autenticación y Formularios

- **React Hook Form**  - Validación de formularios [Fase 3]
- **localStorage**  - Persistencia de sesión [Fase 3]

Peticiones HTTP

- **Axios 1.15.2** - Cliente HTTP

Estilos

- **CSS3** - Grid, Flexbox, Animaciones
- **CSS Variables** - Temas y configuración global

Herramientas de Desarrollo

- **ESLint** - Linting de código
- **Redux DevTools Chrome Extension** - Debugging de estado

Instalación y Setup v2.0

1. Clonar y dependencias

```
git clone https://github.com/AnaLauDB/Art-API.git
cd Art-API
npm install
```

2. Extensión Redux DevTools (Recomendado)

Para debuggear Redux en tiempo real, instala la extensión:

- **Chrome:** [Redux DevTools](#)
- **Firefox:** [Redux DevTools](#)

3. Iniciar desarrollo

```
npm run dev
```

La aplicación estará en <http://localhost:5174>

4. Validar Redux

Una vez iniciada:

1. Abre DevTools (F12)
2. Ve a la pestaña **Redux** (si instalaste la extensión)
3. Deberías ver las actions y el estado global



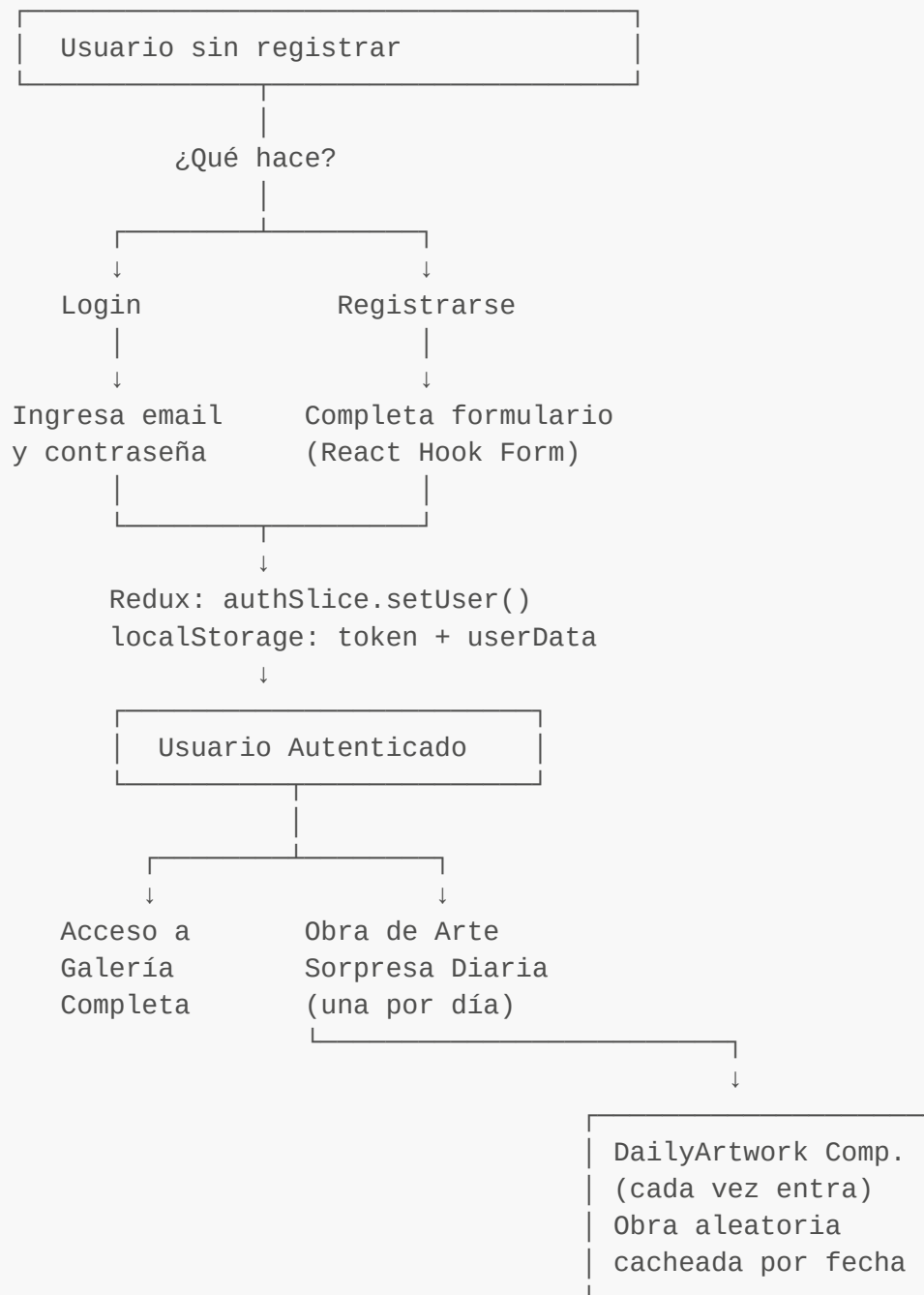
Scripts Disponibles

```
# Desarrollo
npm run dev           # Inicia servidor de desarrollo

# Producción
npm run build         # Compilar para producción
npm run preview       # Previsualizar build
```

```
# Calidad de código
npm run lint          # Ejecutar ESLint
```

🔗 Flujo de Autenticación v2.0 [Fase 3-5]



PROF

Características de Autenticación

- **Login/Registro:** Formularios validados con React Hook Form
- **Persistencia:** Token guardado en localStorage
- **Rutas Protegidas:** Acceso limitado a usuarios logueados
- **Obra Diaria:** Cada usuario recibe una obra sorpresa al ingresar

- **Estado Global:** Redux para mantener datos de sesión

Responsive Design

La aplicación es totalmente responsive:

- **Desktop:** Barra lateral + galería principal
- **Tablet:** Layout adaptado (600px+)
- **Mobile:** Stack vertical optimizado (480px+)

Ejemplos de Búsqueda

- **Artista:** "Pablo Picasso", "Leonardo da Vinci"
- **Técnica:** "óleo", "escultura", "fotografía"
- **Tema:** "paisaje", "retrato", "naturaleza muerta"
- **Período:** Combina años (ej: 1890-1900)

Características API

Búsqueda Semántica

```
// Buscar con términos combinados
await searchArtworks("impressionism painting");

// Con filtros avanzados
await getArtworksByFilters({
  query: "portrait",
  artist: "Rembrandt",
  medium: "oil",
  yearFrom: "1630",
  yearTo: "1669"
});
```

PROF

Gestión de Imágenes

```
// URLs de imágenes optimizadas
getImageUrl(imageId, "small")    // 150x150
getImageUrl(imageId, "medium")   // 400x400
getImageUrl(imageId, "large")    // 843x843
```

Configuración

Variables de Entorno

No requiere configuración especial. La API es pública y accesible sin autenticación.

Build

El proyecto usa Vite con React. La configuración está en `vite.config.js`.



Troubleshooting

"No se pudieron cargar las obras de arte"

- Verifica tu conexión a Internet
- Comprueba que la API está disponible: <https://api.artic.edu/api/v1/artworks/search>
- Revisa la consola del navegador para ver el error específico

Imágenes no cargan

- Algunas obras antiguas pueden no tener imágenes digitalizadas
- La aplicación mostrará un placeholder en estos casos

Rendimiento lento

- Reduce el número de resultados por página
- Usa filtros más específicos para limitar resultados



Objetivos v2.0 - En Desarrollo

Completar en Fase 2-6

- ☒ Redux Setup y State Management
- ☐ Error Boundaries (Global, Regional, Local)
- ☐ Autenticación con Login/Registro
- ☐ Optimización de Imágenes con Lazy Loading
- ☐ Sistema de Obra de Arte Diaria
- ☐ Migración completa a Redux
- ☐ Testing de componentes

PROF

Funcionalidades Adicionales

- ☐ Guardar obras favoritas en localStorage/Redux
- ☐ Historial de búsquedas por usuario
- ☐ Perfil de usuario (pendiente backend)
- ☐ Notificaciones de nuevas obras
- ☐ Compartir obras en redes sociales
- ☐ Modo oscuro/claro



Documentación de Desarrollo

Para Colaboradores

Esta es una aplicación de **nivel Junior/Aprendiz** diseñada para:

- Aprender Redux y manejo de estado global

- Comprender React Hooks profundamente
- Implementar Error Boundaries
- Trabajar con APIs externas
- Optimizar performance en galerías grandes

Convenciones de Código

- **Componentes:** PascalCase
- **Archivos:** PascalCase (componentes), camelCase (services/utils)
- **Redux:** Usar Redux Toolkit con createSlice
- **Comentarios:** Explicar el "por qué", no el "qué"

Debugging

1. **Redux DevTools:** Inspecciona el estado en tiempo real
2. **React DevTools:** Valida renderizaciones innecesarias
3. **Console:** Logs de errores y seguimiento
4. **Network:** Monitorea llamadas a la API

Este proyecto utiliza datos del Art Institute of Chicago bajo su política de Open Access.



Contribuciones

Las contribuciones son bienvenidas. Por favor:

1. Fork el repositorio
2. Crea una rama para tu feature (`git checkout -b feature/AmazingFeature`)
3. Commit tus cambios (`git commit -m 'Add some AmazingFeature'`)
4. Push a la rama (`git push origin feature/AmazingFeature`)
5. Abre un Pull Request



Autor

Ana Lau

- GitHub: [@AnaLauDB](#)



Soporte

Si encuentras problemas o tienes sugerencias, por favor:

- Abre un issue en GitHub
- Contacta a través de GitHub Issues



Agradecimientos

- Art Institute of Chicago por proporcionar la API pública
- Comunidad de React y Vite
- Todos los contribuidores